

Cooperation and Exploitation in LLM Policy Synthesis for Sequential Social Dilemmas

Anonymous authors

Paper under double-blind review

Abstract

We study *LLM policy synthesis*: using a large language model to iteratively generate programmatic agent policies for multi-agent environments. Rather than training neural policies via reinforcement learning, our framework prompts an LLM to produce Python policy functions, evaluates them in self-play, and refines them using performance feedback across iterations. We investigate *feedback engineering* (the design of what evaluation information is shown to the LLM during refinement) comparing *sparse feedback* (scalar reward only) against *dense feedback* (reward plus social metrics: efficiency, equality, sustainability, peace). Across two canonical Sequential Social Dilemmas (Gathering and Cleanup) and two frontier LLMs (Claude Sonnet 4.6, Gemini 3.1 Pro), dense feedback consistently matches or exceeds sparse feedback on all metrics. The advantage is largest in the Cleanup public goods game, where providing social metrics helps the LLM calibrate the costly cleaning–harvesting tradeoff. Rather than triggering over-optimization of fairness, social metrics serve as a *coordination signal* that guides the LLM toward more effective cooperative strategies, including territory partitioning, adaptive role assignment, and the avoidance of wasteful aggression. We further perform an adversarial experiment to determine whether LLMs can reward hack these environments. We characterize five attack classes and discuss mitigations, highlighting an inherent tension in LLM policy synthesis between expressiveness and safety.

Code at <https://github.com/vicgalle/llm-policies-social-dilemmas>.

1 Introduction

Sequential Social Dilemmas (SSDs) (Leibo et al., 2017) are multi-agent environments where individually rational behavior leads to collectively suboptimal outcomes, e.g., they are the multi-agent analog of the prisoner’s dilemma, extended to temporally rich Markov games. Standard multi-agent reinforcement learning (MARL) struggles with SSDs due to credit assignment difficulties, non-stationarity, and the vast joint action space (Busoniu et al., 2008).

Recent advances in large language models (LLMs) open a fundamentally different approach: rather than learning policies through gradient-based optimization in *parameter space*, an LLM can directly *synthesize programmatic policies* in *algorithm space*: writing executable code that implements complex coordination strategies such as territory division, role assignment, and conditional cooperation. This paradigm, related to FunSearch (Romera-Paredes et al., 2024) and Eureka (Ma et al., 2024), sidesteps the sample efficiency bottleneck of MARL entirely: a single LLM generation step can produce a sophisticated coordination algorithm that would require millions of RL episodes to discover.

A critical question arises when using iterative LLM synthesis: *what feedback should the LLM receive between iterations?* The intuitive assumption is that richer feedback enables better policies: showing the LLM explicit social metrics (equality, sustainability, peace) should help it navigate social dilemmas. We test this hypothesis across two frontier LLMs and two canonical SSDs, and find that the intuition is correct: providing dense social feedback consistently matches or exceeds sparse scalar reward.

The mechanism is that social metrics act as a **coordination signal**. In the Gathering environment, equality information helps the LLM discover that territory partitioning eliminates wasteful competition, and that aggression is counterproductive. In Cleanup, sustainability and equality metrics help the LLM calibrate the number of agents assigned to the costly but socially necessary cleaning role, yielding up to 54% higher efficiency than sparse feedback. Across both games and both LLMs, dense feedback also produces higher equality and sustainability without sacrificing efficiency.

Contributions.

- We formalize *iterative LLM policy synthesis* for multi-agent SSDs, where an LLM generates Python policy functions evaluated in self-play and refined through feedback (Section 2).
- We introduce *feedback engineering* as a design axis, comparing sparse (reward-only) vs. dense (reward + social metrics) feedback (Section 2.3).
- Across two SSDs and two frontier LLMs, we show that dense feedback consistently matches or exceeds sparse feedback on all metrics, with social metrics serving as a coordination signal rather than a distraction (Section 3).
- We identify and characterize *direct environment mutation* attacks, a class of reward hacking where LLM-generated policies exploit the mutable environment reference to bypass game mechanics entirely (Section 4).

2 Framework

2.1 Sequential Social Dilemmas

An SSD is a partially observable Markov game

$$\mathcal{G} = \langle N, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$$

with N agents, state space $\mathcal{S}$ (the gridworld configuration), per-agent action spaces $\mathcal{A}_i$, transition function T , reward functions R_i , and episode horizon H . We study two canonical SSDs:

Gathering (Leibo et al., 2017). Agents navigate a 2D gridworld and collect apples (+1 reward). Apples respawn on a fixed 25-step timer. Agents may fire a tagging beam (2 hits remove a rival for 25 steps). The dilemma: agents can coexist peacefully and share resources, or attack rivals to monopolize apples, but aggression wastes time and reduces total welfare.

Cleanup (Hughes et al., 2018). A public goods game with two regions: a river that accumulates waste, and an orchard where apples grow. Apples only regrow when the river is sufficiently clean. Agents can fire a cleaning beam (costs -1) to remove waste, or collect apples (+1). A penalty beam (costs -1 , inflicts -50 on the target) can tag rivals out for 25 steps. The dilemma: cleaning is costly but benefits everyone; purely selfish agents free-ride on others’ cleaning.

Both games use 8–9 discrete actions (4 movement directions, 2 rotations, beam, stand, and optionally clean) and episodes of $H = 1000$ steps. Screenshots of both environments are shown in Figure 4 (Appendix).

Following Perolat et al. (2017), we evaluate outcomes using four social metrics. Let $R_i = \sum_{t=0}^{H-1} r_i^t$ denote agent i ’s episode return. Then:

$$\text{Efficiency: } U = \frac{1}{H} \sum_{i=1}^N R_i \quad (1)$$

$$\text{Equality: } E = 1 - \frac{\sum_{i,j} |R_i - R_j|}{2N \sum_i R_i} \quad (2)$$

$$\text{Sustainability: } S = \frac{1}{N} \sum_{i=1}^N \bar{r}_i \quad (3)$$

$$\text{Peace: } P = \frac{1}{H} \sum_{t=0}^{H-1} |\{i : \text{active}_i^t\}| \quad (4)$$

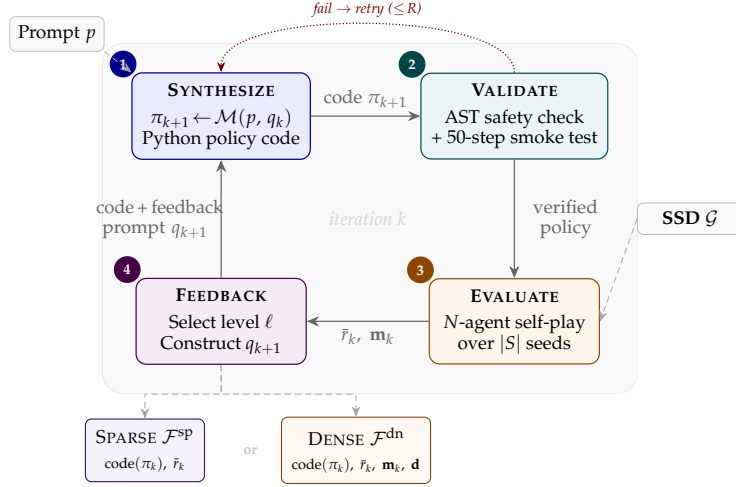


Figure 1: Iterative LLM policy synthesis framework (Algorithm 1). At each iteration k , the LLM SYNTHESIZES (1) a Python policy from the system prompt p and previous feedback, which is VALIDATED (2) via AST checks and a smoke test (retrying on failure up to R times), EVALUATED (3) in N -agent self-play, and the results packaged as either *sparse* or *dense* FEEDBACK (4).

81 where $\bar{t}_i$ is the mean timestep at which agent i collects positive reward (higher means
82 resources remain available later), and active_i^t indicates agent i is not tagged out at step t .

83 2.2 Iterative LLM Policy Synthesis

84 Let Π denote the space of *programmatic policies*: deterministic functions $\pi : S \times [N] \rightarrow \mathcal{A}$
85 expressed as executable Python code. Each policy has access to the full environment state
86 and a library of helper functions: breadth-first search (BFS) pathfinding, beam targeting,
87 and coordinate transforms. This state access is a deliberate design choice: programmatic
88 policies operate in algorithm space rather than in the reactive observation-to-action space of
89 neural policies. Code as Policies (Liang et al., 2023) demonstrated that LLMs can generate
90 executable robot policy code that processes perception outputs and parameterizes control
91 primitives via few-shot prompting. And Eureka (Ma et al., 2024) uses LLMs to generate
92 code-based reward functions (rather than policies) from the environment source code. Our
93 work differs from these in that the LLM iteratively synthesizes complete agent policies for
94 a multi-agent setting, where the generated code must simultaneously coordinate across
95 agents sharing the same program.

96 A frozen LLM $\mathcal{M}$ acts as a *policy synthesizer*. Given a system prompt p describing the
97 environment API and a feedback prompt q_k , it generates source code implementing a new
98 policy:

$$\pi_{k+1} = \mathcal{M}\left(p, q\left(\pi_k, \mathcal{F}_k^\ell\right)\right) \quad (5)$$

99 where π_k is the previous policy (its source code), $\mathcal{F}_k^\ell$ is the evaluation feedback at level ℓ ,
100 and $q(\cdot)$ constructs the user prompt.

101 **Self-play evaluation.** All N agents execute the same policy π_k (homogeneous self-play).
102 The evaluation computes feedback over a set of random seeds S :

$$\mathcal{F}_k = \text{Eval}(\underbrace{\pi_k, \dots, \pi_k}_N; \mathcal{G}, S) = (\bar{r}_k, \mathbf{m}_k) \quad (6)$$

103 where $\bar{r}_k = \frac{1}{N|S|} \sum_{s \in S} \sum_i R_i^{(s)}$ is the mean per-agent return and $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$ is the
104 social metrics vector.

Algorithm 1: Iterative LLM Policy Synthesis**Input:** Game $\mathcal{G}$, LLM $\mathcal{M}$, system prompt p , iterations K , feedback level ℓ , eval seeds S **Output:** Final policy π_K

```

1  $\pi_0 \leftarrow \mathcal{M}(p, \text{"generate initial policy"})$ 
2  $\mathcal{F}_0^\ell \leftarrow \text{Eval}(\pi_0; \mathcal{G}, S)$ 
3 for  $k = 1, \dots, K$  do
4   for attempt = 1, ...,  $R$  do
5      $\pi_k \leftarrow \mathcal{M}(p, q(\pi_{k-1}, \mathcal{F}_{k-1}^\ell))$ 
6     if  $\text{Validate}(\pi_k)$  then break
7      $q \leftarrow q \parallel \text{error\_msg}$  // append for retry
8    $\mathcal{F}_k^\ell \leftarrow \text{Eval}(\pi_k; \mathcal{G}, S)$ 
9 return  $\pi_K$ 

```

105 **Validation.** Each generated policy undergoes AST-based safety checking (blocking danger-
 106 ous operations such as eval, file I/O, and network access) followed by a 50-step smoke test
 107 to catch runtime errors. If validation fails, the error message is appended to the prompt and
 108 generation is retried (up to 3 attempts).

109 2.3 Feedback Engineering

110 We define two feedback levels ℓ that control what information the LLM receives between
 111 iterations:

112 **Sparse feedback** (REWARD-ONLY). The LLM receives the previous policy’s source code and
 113 the scalar mean per-agent reward:

$$\mathcal{F}_k^{\text{sp}} = (\text{code}(\pi_k), \bar{r}_k) \quad (7)$$

114 **Dense feedback** (REWARD+SOCIAL). The LLM additionally receives the full social metrics
 115 vector together with natural-language definitions of each metric:

$$\mathcal{F}_k^{\text{dn}} = (\text{code}(\pi_k), \bar{r}_k, \mathbf{m}_k, \mathbf{d}) \quad (8)$$

116 where $\mathbf{d}$ contains textual definitions (e.g., “Equality: fairness of reward distribution, 1.0 =
 117 perfectly equal”). We avoid leaking environment information in these definitions, to ensure a
 118 fair comparison between methods.

119 In both modes, the system prompt instructs the LLM to *maximize per-agent reward*: the social
 120 metrics in dense feedback are presented as informational context, not explicit optimization
 121 targets. Both modes use the neutral framing “all agents run the same code” (no adversarial
 122 language, nor placing emphasis on cooperation).

123 The full procedure is illustrated in Figure 1 and summarized in Algorithm 1. At iteration 0
 124 the LLM generates a policy from scratch (no prior code); subsequent iterations receive the
 125 previous policy’s code and feedback. The key design question is whether $\ell = \text{sp}$ or $\ell = \text{dn}$
 126 produces better policies.

127 3 Experiments

128 3.1 Setup

129 We run both SSDs with $N = 10$ agents on large map variants. Gathering uses a 38×16
 130 gridworld with ~ 120 apple spawns; Cleanup uses a gridworld with separate river and
 131 orchard regions. We run $K = 3$ refinement iterations per configuration, evaluating each
 132 policy over $|S| = 5$ random seeds. Each configuration is repeated over 3 independent runs
 133 (different random seeds and LLM sampling).

Table 1: Results across two SSDs, two LLMs, and three feedback configurations. LLM values show the mean over 3×5 independent runs (min-max in parentheses). U : efficiency (Eq. 1). E : equality (Eq. 2). S : sustainability (Eq. 3). Bold marks the best value per game $\times$ model block. Baselines (bottom of each game block) are non-LLM methods for reference.

Game	Model	Feedback	U	E	S
Gathering	Claude Sonnet 4.6	ZERO-SHOT	1.85 (1.52–2.02)	0.52 (0.35–0.63)	298.6 (180–374)
		REWARD-ONLY	3.47 (3.39–3.56)	0.72 (0.61–0.87)	402.9 (314–500)
		REWARD+SOCIAL	3.53 (1.60–4.58)	0.84 (0.63–0.94)	452.7 (356–502)
	Gemini 3.1 Pro	ZERO-SHOT	3.71 (1.90–4.46)	0.79 (0.35–0.96)	443.2 (180–504)
		REWARD-ONLY	4.58 (4.48–4.63)	0.97 (0.97–0.97)	502.5 (502–503)
		REWARD+SOCIAL	4.59 (4.50–4.65)	0.97 (0.97–0.97)	502.7 (501–505)
	GEPA (Gemini 3.1 Pro)		3.45 (3.16–3.95)	0.91 (0.83–0.96)	496.2 (491–499)
	Q-learner		0.77 (0.73–0.81)	0.83 (0.81–0.85)	508.2 (504–513)
	BFS Collector		1.29	0.54	489.5
Cleanup	Claude Sonnet 4.6	ZERO-SHOT	−1.01 (−7.90–0.93)	−3.06 (−15.9–1.00)	137.0 (19–214)
		REWARD-ONLY	1.14 (1.04–1.32)	−0.47 (−0.66–−0.11)	233.0 (225–245)
		REWARD+SOCIAL	1.37 (1.16–1.73)	0.09 (−0.15–0.55)	294.6 (209–456)
	Gemini 3.1 Pro	ZERO-SHOT	0.45 (−2.32–1.28)	−0.45 (−1.38–0.73)	274.1 (158–421)
		REWARD-ONLY	1.79 (1.16–2.57)	0.13 (−0.47–0.47)	386.0 (186–503)
		REWARD+SOCIAL	2.75 (1.44–3.49)	0.54 (0.33–0.67)	432.6 (297–501)
	GEPA (Gemini 3.1 Pro)		0.77 (0.58–1.07)	−1.75 (−2.54–−0.62)	209.5 (184–260)
	Q-learner		−0.16 (−0.28–−0.06)	0.20 (−0.37–1.00)	208.6 (121–301)
	BFS Collector		0.10	0.61	16.4

Models. We evaluate two frontier LLMs: **Claude Sonnet 4.6** (Anthropic) and **Gemini 3.1 Pro** (Google). Both use the highest available thinking budget for chain-of-thought reasoning before code generation.

Baselines. **Q-learner**: tabular Q-learning with a shared Q-table and non-trivial feature engineering: 7 hand-crafted discrete features in Gathering (BFS direction and distance to nearest apple, local apple density, nearest-agent direction and distance, beam-path check, own hit count; 4 320 states) and 8 features in Cleanup (adding BFS to nearest waste, global waste density, and a can-clean check; 11 664 states), plus cooperative reward shaping ($0.5 \cdot r_i + 0.5 \cdot \bar{r}$ – beam penalty, with an additional cleaning bonus in Cleanup). Trained for 1000 episodes with ϵ -greedy exploration. **BFS Collector**: a hand-coded heuristic that performs BFS to the nearest apple, never beams or cleans. **GEPA** (Agrawal et al., 2026): Genetic-Pareto prompt Optimization, an LLM-based meta-optimizer that iteratively refines the *system prompt* (not the policy code) using a reflection LM. We run GEPA with the same Gemini 3.1 Pro model for both generation and reflection, with $K=3$ reflection iterations and $n_{\text{eval}}=5$ evaluation seeds per candidate, matching the computational budget of the iterative code-level methods above. Unlike REWARD-ONLY and REWARD+SOCIAL, GEPA’s reflection LM receives only the scalar reward; social metric definitions are not included in the prompt to avoid information leakage.

Configurations. For each model $\times$ game, we compare three settings: (1) ZERO-SHOT: the zero-shot initial policy generated by the LLM (no refinement); (2) REWARD-ONLY: $K=3$ iterations with sparse feedback; (3) REWARD+SOCIAL: $K=3$ iterations with dense feedback.

3.2 Main Results

Table 1 presents results across both games, both models, and all feedback configurations. Three findings emerge.

Finding 1: LLM policy synthesis dominates traditional methods. Both feedback modes produce large improvements over the ZERO-SHOT baseline, and all refined LLM policies

dramatically outperform non-LLM baselines. In Gathering, the best LLM configuration (Gemini, dense, $U=4.59$) achieves $6.0\times$ the Q-learner ($U=0.77$) and $3.6\times$ the BFS heuristic ($U=1.29$). In Cleanup, the gap is even larger: $U=2.75$ vs. -0.16 for Q-learning, which fails entirely at the credit assignment required for the cleaning–harvesting tradeoff. Iterative refinement is key: Claude’s ZERO-SHOT achieves $U=-1.01$ in Cleanup (agents lose reward on average), while 3 iterations push efficiency to $U=1.14$ – 1.37 .

Finding 1b: Code-level feedback outperforms prompt-level optimization. GEPA optimizes the system prompt rather than the policy code, using the same model (Gemini 3.1 Pro) and comparable budget ($K=3$ iterations). In Gathering, GEPA achieves $U=3.45$ —above Claude’s best but 25% below Gemini’s direct code-level iteration ($U=4.59$). In Cleanup the gap widens dramatically: $U=0.77$ vs. 2.75 ($3.6\times$ lower), with severely negative equality ($E=-1.75$) indicating free-riding. This confirms that direct code-level feedback, where the LLM sees and revises its own policy source, is substantially more effective than prompt-level meta-optimization for discovering cooperative strategies in social dilemmas.

Finding 2: Dense feedback consistently matches or exceeds sparse feedback. Across all four game $\times$ model combinations, REWARD+SOCIAL (dense) achieves equal or higher efficiency than REWARD-ONLY (sparse).

The advantage is most pronounced in Cleanup, where the cleaning–harvesting coordination problem benefits from explicit social metrics. With Gemini, dense feedback yields 54% higher efficiency than sparse ($U: 2.75$ vs. 1.79). With Claude, the gain is 20% ($U: 1.37$ vs. 1.14). In Gathering, where the coordination challenge is simpler (agents need only avoid competing for the same apples), the two modes perform similarly, with dense feedback holding a slight edge for Claude ($U: 3.53$ vs. 3.47).

Finding 3: Social metrics serve as a coordination signal. Dense feedback improves not only efficiency but also equality and sustainability, simultaneously, without tradeoffs. In Cleanup with Gemini, dense feedback raises equality from $E=0.13$ to 0.54 and sustainability from $S=386$ to 433 , while also achieving the highest efficiency ($U=2.75$).

Examining the generated policies reveals how this occurs (Appendix A). Under dense feedback, the LLM develops *waste-adaptive cleaner schedules* that scale the number of cleaning agents with river pollution level (up to 7 of 10 agents), combined with sophisticated beam positioning that maximizes waste removal per shot (Appendix A.2). Under sparse feedback, the LLM instead assigns fixed cleaning roles to a small subset of agents, producing less adaptive and less effective strategies.

In Gathering, dense feedback leads to *BFS-Voronoi territory partitioning*—a multi-source flood-fill that assigns each apple to the nearest agent by true shortest-path distance—with zero aggression (Appendix A.1). Sparse feedback discovers column-strip territory division but also develops multi-tier combat systems (beaming, chasing wounded opponents) that waste actions and reduce peace.

Cross-model comparison. Gemini 3.1 Pro consistently outperforms Claude Sonnet 4.6 across all configurations. In Gathering, Gemini achieves near-optimal performance ($U\approx 4.6$, $E=0.97$) with both feedback modes, while Claude reaches $U\approx 3.5$ with higher variance across runs. In Cleanup, the gap is larger: Gemini’s best configuration ($U=2.75$) doubles Claude’s ($U=1.37$). Gemini also shows lower variance across runs (e.g., $U: 4.50$ – 4.65 vs. 1.60 – 4.58 in Gathering with dense feedback), suggesting more consistent policy synthesis.

3.3 Why Dense Feedback Helps: Feedback Aliasing

The asymmetry in Table 1 (dense feedback yields much higher efficiency in Cleanup than in Gathering) calls for explanation. We argue that the key factor is *feedback aliasing*: whether scalar reward alone is sufficient to diagnose coordination failures and determine the correct refinement direction.

In Cleanup, the central coordination parameter is the number of cleaners n_c . Too few cleaners and waste overwhelms the river, choking apple regrowth; too many and the cleaning cost exceeds the marginal regrowth benefit. Total reward is therefore a concave function of n_c with a unique interior optimum: increasing in n_c while cleaners still reduce waste meaningfully, and decreasing once the river is already clean and additional cleaners only add cost. Crucially, for any suboptimal reward level below this peak, there are *two* failure modes that produce the same scalar reward: under-cleaning (too few cleaners, resources depleted) and over-cleaning (too many cleaners, excessive cost), but require opposite corrections. Scalar reward alone cannot distinguish them. Social metrics break this alias: under-cleaning manifests as low sustainability (apples vanish early due to pollution), while over-cleaning manifests as low equality (cleaners bear disproportionate cost while few harvesters capture most reward). Dense feedback thus gives the LLM a diagnostic signal that sparse feedback lacks, enabling it to identify *which* failure mode it faces and adjust accordingly. This is visible in the generated policies: under dense feedback, the LLM develops waste-adaptive schedules that scale n_c with pollution level (Listing 3), while under sparse feedback it settles on fixed allocations that cannot self-correct (Listing 4).

Gathering lacks this structure. There is no costly pro-social action, so the coordination problem reduces to a single axis: reducing territorial overlap. As overlap decreases, both per-agent reward and equality improve monotonically: there is no tradeoff, no interior optimum, and no second failure mode to confuse with the first. Scalar reward is therefore sufficient to determine the refinement direction (less competition $\rightarrow$ more reward), and dense feedback provides little additional signal. This explains why both feedback modes converge to similar performance in Gathering, with Gemini reaching near-optimal territory partitioning under either mode.

4 Case Study: Reward Hacking

Our framework gives policies programmatic access to the environment object (Section 2.2). A natural question is whether LLMs can exploit this access. We prompted Claude Opus 4.6 to analyze the environment and generate reward-hacking policies: it produced five distinct attacks autonomously, without guidance beyond the initial request. This demonstrates that the same models used for cooperative policy synthesis (Section 3) can equally discover exploits when prompted in an adversarial way.

Mechanism. Each policy is called as $a_i = \pi(e, i)$ where e is the live environment instance. While our AST validator blocks dangerous operations (eval, file I/O, network access), it cannot distinguish attribute *reads* from *writes*: any NumPy array attribute (agent_pos, apple_alive, waste, agent_timeout) is silently mutable, and no integrity check runs between the policy call and env.step().

The LLM identified two attack classes: *state manipulation* (teleporting onto apples, disabling rivals by setting their timeout to ∞) and *dynamics bypass* (clearing waste or force-spawning apples every step). All five attacks pass AST validation and the smoke test: they are valid `policy(env, agent_id) -> int` functions indistinguishable from legitimate policies at the interface level.

Results. Table 2 reports results on the same Cleanup configuration as Table 1. Dynamics bypass attacks are dramatically more powerful than state manipulation: against BFS victims, teleporting yields only $2\times$ amplification (the bottleneck is apple respawn, not pathfinding), while force-spawning apples reaches the per-step theoretical maximum ($59\times$). Interestingly, better victims can *amplify* certain attacks: against optimized agents that actively clean waste, teleporting jumps from $2\times$ to $9.6\times$ because the attacker free-rides on their cleaning. Conversely, disabling optimized victims collapses the ecosystem (removing the cleaners leaves the attacker alone in a polluted map).

The most concerning finding connects directly to Section 3: dynamics bypass attacks that benefit all agents (purge waste, spawn apples) actually *improve* measured social metrics. Against BFS victims, “spawn apples” achieves the highest efficiency ($U = 5.99$) and sus-

Table 2: Reward hacking via environment mutation in Cleanup ($N = 10$, large map, 1000 steps). Agent 0 runs the attack; agents 1–9 play the victim policy. Each cell shows Agent 0’s reward and amplification over baseline.

Class	Attack	vs BFS	vs Optimized
—	Baseline	17 (1.0 $\times$)	104 (1.0 $\times$)
I: State	Teleport	34 (2.0 $\times$)	1000 (9.6 $\times$)
	Disable rivals	103 (6.1 $\times$)	103 (1.0 $\times$)
II: Dynamics	Purge waste	765 (45 $\times$)	765 (7.4 $\times$)
	Spawn apples	999 (59 $\times$)	999 (9.6 $\times$)
	Combined	1000 (59 $\times$)	1000 (9.6 $\times$)

tainability ($S = 500.5$) of any configuration—surpassing every LLM-synthesized policy in Table 1. This illustrates a Goodharting (Goodhart, 1984) risk: a metric-optimizing LLM could discover dynamics manipulation as a “legitimate” strategy, as it maximizes social metrics while fundamentally violating the game’s intended mechanics.

Implications. Standard mitigations exist (read-only proxies, state hashing, process isolation) but they highlight a deeper tension. The expressiveness that enables the BFS pathfinding and territory partitioning strategies of Section 3 is the same access that enables exploitation. Designing policy interfaces that are expressive enough for sophisticated coordination yet resistant to reward hacking remains an open challenge, and any verification pipeline must assume adversarial capability at least equal to the synthesizer’s.

5 Related Work

Sequential Social Dilemmas. Leibo et al. (2017) introduced SSDs as temporally extended Markov games exhibiting cooperation–defection tension, instantiated in the Gathering gridworld. Hughes et al. (2018) proposed the Cleanup game as a public goods variant requiring costly pro-social labor. Perolat et al. (2017) formalized social outcome metrics (efficiency, equality, sustainability, peace) for evaluating multi-agent cooperation in SSDs.

LLMs for policy and program synthesis. FunSearch (Romera-Paredes et al., 2024) uses LLMs to iteratively evolve programs that solve combinatorial optimization problems. Eureka (Ma et al., 2024) applies LLM code generation to design reward functions for robot control. Voyager (Wang et al., 2024) generates executable skill code for embodied agents. Code as Policies (Liang et al., 2023) generates executable robot policy code from natural language via few-shot prompting; we extend this to multi-agent settings with iterative performance-driven refinement rather than one-shot instruction following. ReEvo (Ye et al., 2024) evolves heuristic algorithms through LLM reflection. Our work differs in applying LLM program synthesis to *multi-agent* environments, where policies must coordinate across agents sharing the same code.

LLM reflection and feedback. Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023) demonstrate that LLMs can self-improve through verbal feedback loops. OPRO (Yang et al., 2024) frames optimization as iterative prompt refinement. Shi et al. (2026) introduce Experiential Reinforcement Learning (ERL), a training paradigm that embeds an experience–reflection–consolidation loop into reinforcement learning, where the model reflects on failed attempts and internalizes corrections via self-distillation. GEPA (Agrawal et al., 2026) combines reflective natural-language feedback with Pareto-based evolutionary search to optimize prompts, demonstrating that structured reflection on execution traces can outperform RL with substantially fewer rollouts. Our work specifically investigates how the *content* of evaluation feedback (scalar reward vs. multi-objective social metrics) affects the quality of LLM-generated multi-agent policies: what we call *feedback engineering*.

Reward hacking. When optimizing agents exploit unintended shortcuts in the reward signal or environment implementation, the resulting *reward hacking* (Skalse et al., 2022) can produce high-scoring but undesirable behavior. Pan et al. (2022) demonstrate that even small misspecifications in reward functions lead to qualitatively wrong policies. Goodhart’s Law (Goodhart, 1984) (“when a measure becomes a target, it ceases to be a good measure”) formalizes this risk. Our adversarial analysis (Section 4) identifies a novel instantiation: LLM-generated policies that exploit environment state, a vulnerability absent from standard RL pipelines where the agent–environment boundary is enforced at the API level. Gallego (2025) address in-context reward hacking at the specification level. This is complementary to our Section 4 finding that LLM-generated policies can exploit mutable environment state: SSC corrects the objective, while our analysis identifies the need to secure the environment interface.

6 Discussion and Conclusion

We have shown that LLM policy synthesis is a powerful approach to multi-agent coordination in SSDs, with iterative refinement producing substantial improvements over zero-shot generation. Two complementary findings emerge.

First, *richer feedback helps*: dense social metrics consistently match or exceed sparse scalar reward across two games and two frontier LLMs. Rather than being a distraction or source of over-optimization, social metrics serve as a *coordination signal* that helps the LLM understand the structure of the game. In Cleanup, sustainability feedback guides the LLM toward allocating sufficient cleaning resources; equality feedback encourages more balanced role assignments. In Gathering, peace and equality information steers the LLM away from wasteful aggression toward pure cooperation. This is consistent with the broader literature on LLM reflection (Shinn et al., 2023; Madaan et al., 2023): providing structured, multi-dimensional evaluation enables more targeted self-improvement than scalar feedback alone. Our contribution is showing that this extends to multi-agent settings where the feedback dimensions capture social outcomes.

Second, *expressiveness enables exploitation*. The direct environment mutation attacks in Section 4 demonstrate that the same full-state access enabling BFS-Voronoi territory partitioning and waste-adaptive cleaning also permits reward hacking. Critically, the most powerful attacks (dynamics bypass) also improve all measured social metrics, demonstrating a Goodharting risk (Goodhart, 1984) where metric optimization and intended behavior diverge. This dual-use nature of LLM reasoning in multi-agent settings (the same capability enabling both sophisticated cooperation and sophisticated exploitation) is, we believe, the central challenge for scaling LLM policy synthesis beyond controlled experiments.

Limitations. Our SSDs are relatively small-scale; scaling to larger, more complex environments remains future work. The adversarial analysis (Section 4) demonstrates autonomous attack discovery from a single prompted LLM; future work should evaluate whether attacks emerge organically during the iterative synthesis loop itself (Algorithm 1) without explicit adversarial prompting, and whether defenses hold under adaptive attackers.

Future work. Promising directions include: (1) studying intermediate feedback levels (e.g., showing efficiency but not equality), (2) extending to heterogeneous policies (different code per agent), (3) designing policy interfaces that balance expressiveness with tamper-resistance (Section 4), and (4) combining LLM policy synthesis with neural policy distillation for deployment under partial observability.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnab Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026. URL <https://arxiv.org/abs/2507.19457>.

- Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008. doi: 10.1109/TSMCC.2007.913919.
- Víctor Gallego. Specification self-correction: Mitigating in-context reward hacking through test-time refinement. In *The 1st Workshop on Test-time Scaling and Reasoning Models*, 2025. URL <https://openreview.net/forum?id=UU9KCA0sTH>.
- Charles AE Goodhart. Problems of monetary management: The UK experience. *Monetary Theory and Practice*, pp. 91–121, 1984.
- Edward Hughes, Joel Z. Leibo, Matthew Phillips, Karl Tuyls, Edgar A. Duéñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R. McKee, R. Koster, Heather Roff, and Thore Graepel. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*, 2018. URL <https://api.semanticscholar.org/CorpusID:52843325>.
- Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*, pp. 464–473, 2017.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, 2023. doi: 10.1109/ICRA48891.2023.10160591.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=IEduRU055F>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations*, 2022.
- Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. A multi-agent reinforcement learning model of common-pool resource appropriation. *Advances in neural information processing systems*, 30, 2017.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Taiwei Shi, Sihao Chen, Bowen Jiang, Linxin Song, Longqi Yang, and Jieyu Zhao. Experiential reinforcement learning. *arXiv preprint arXiv:2602.13949*, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=vAE1hFckW6>.
- Joar Max Viktor Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=yb3H0X031X2>.

- 399 Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi
 400 Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large
 401 language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL
 402 <https://openreview.net/forum?id=ehfRiF0R3a>.
- 403 Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and
 404 Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference*
 405 *on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Bb4VGOWELI>.
- 406 Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim,
 407 Jinkyoo Park, and Guojie Song. Reevo: Large language models as hyper-heuristics with
 408 reflective evolution. In *The Thirty-eighth Annual Conference on Neural Information Processing*
 409 *Systems*, 2024. URL <https://openreview.net/forum?id=483IPG0HWL>.

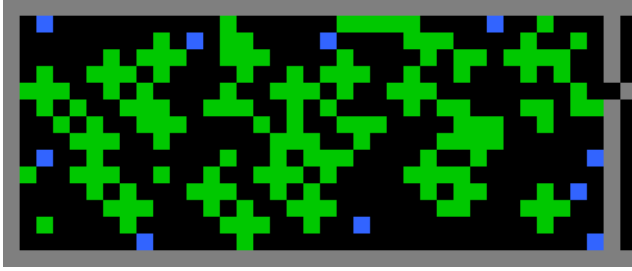


Figure 2: *
(a) Gathering

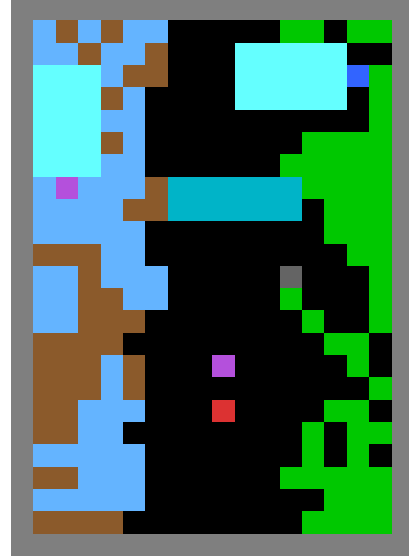


Figure 3: *
(b) Cleanup

Figure 4: Screenshots of the two SSD environments used in our experiments. **(a) Gathering:** agents (colored markers) navigate a gridworld to collect apples (green cells); apples respawn on a fixed timer and agents may fire tagging beams to temporarily remove rivals. **(b) Cleanup:** agents operate in two regions: a river accumulating waste (brown) and an orchard where apples grow (green); agents must cooperatively clean the river for apples to regrow. Agents may fire beams (cyan) to tag rivals out.

410 A Generated Policy Analysis

411 All code excerpts below are verbatim LLM output, extracted from the best-performing
 412 iteration of representative runs. Comments in [brackets] are ours.

413 A.1 Gathering: Territory Strategies

414 Under dense feedback, the LLM discovers *BFS-Voronoi territory partitioning*: a multi-source
 415 flood-fill simultaneously from all alive agents computes true shortest-path ownership of
 416 every cell, correctly handling walls where Manhattan distance fails. The policy is purely
 417 cooperative—no agent ever fires the tagging beam.

Listing 1: Gathering — dense feedback (BFS-Voronoi, zero aggression).

```
418 # [Multi-source BFS: O(H*W) flood-fill assigns territory]
```

```

420 | bfs_q = deque()
421 | dist_map = {}
422 |
423 | for i in range(env.n_agents):
424 |     if int(env.agent_timeout[i]) > 0:
425 |         continue
426 |     r, c = int(env.agent_pos[i][0]), int(env.agent_pos[i][1])
427 |     dist_map[(r, c)] = (0, i)
428 |     bfs_q.append((r, c, 0, i))
429 |
430 | while bfs_q:
431 |     r, c, d, owner = bfs_q.popleft()
432 |     cur = dist_map.get((r, c), (10**9, 10**9))
433 |     if d > cur[0] or (d == cur[0] and owner > cur[1]):
434 |         continue
435 |     for dr2, dc2 in ((-1,0),(1,0),(0,-1),(0,1)):
436 |         nr, nc = r + dr2, c + dc2
437 |         if 0<=nr<H and 0<=nc<W and not walls[nr][nc]:
438 |             nd = d + 1
439 |             prev2 = dist_map.get((nr,nc), (10**9, 10**9))
440 |             if nd<prev2[0] or (nd==prev2[0] and owner<prev2[1]):
441 |                 dist_map[(nr,nc)] = (nd, owner)
442 |                 bfs_q.append((nr, nc, nd, owner))
443 |
444 | # Phase 1: BFS-navigate to nearest owned alive apple
445 | # Phase 2: camp at nearest owned dead spawn (wait for respawn)
446 | # Phase 3: global fallback to any reachable alive apple

```

Under sparse feedback, the LLM discovers *column-strip territory* (a simpler $O(1)$ assignment) but also develops a multi-tier combat system that wastes actions on beaming and chasing:

Listing 2: Gathering — sparse feedback (column strips + combat tiers).

```

450 | # [Static column-strip territory: O(1) assignment]
451 | zone_width = env.width / env.n_agents # 38/10 = 3.8
452 | zone_start = int(agent_id * zone_width)
453 | zone_end = int((agent_id+1) * zone_width)
454 |
455 | # [Multi-tier combat system wastes actions]
456 | # Tier 1: Kill shot directly ahead -- always fire
457 | # Tier 2: Rotate once to land kill shot
458 | # Tier 3: Chase wounded opponents within range 8
459 | # Tier 4: First hit on very close targets (range <= 2)
460 |
461 | # [Evasion when one hit from being tagged]
462 | if my_hits >= hits_to_tag - 1 and threats:
463 |     # Flee away from nearest threat
464 |     ...
465 |
466 | # [Territory-based collection: home zone first, global fallback]
467 | home_apples = {pos for pos in alive_apples
468 |                 if zone_start <= pos[1] < zone_end}
469 | result = bfs_to_target_set(env, agent_id, home_apples)
470 |
471 |

```

The BFS-Voronoi policy achieves higher reward than column strips because territory adapts dynamically as agents move, and the absence of combat means every action is spent collecting.

A.2 Cleanup: Cleaner Allocation Strategies

The most impactful difference between dense and sparse policies is in *cleaner allocation*—how many agents are assigned to the costly but socially necessary cleaning role—and *cleaning efficiency*—how effectively each cleaner removes waste.

Under dense feedback, the LLM develops a waste-adaptive scaling schedule combined with optimized beam positioning:

Listing 3: Cleanup — dense feedback (adaptive scaling + optimal beam positioning).

```

481 | # [Waste-adaptive schedule: up to 7/10 agents clean]
482 | if waste_ratio >= 0.8: n_cleaners = 7
483 | elif waste_ratio >= 0.6: n_cleaners = 5
484 | elif waste_ratio >= 0.4: n_cleaners = 3
485 | elif waste_ratio >= 0.2: n_cleaners = 2
486 | elif waste_ratio >= 0.07: n_cleaners = 1
487 |

```

```

488     else:                                     n_cleaners = 0
489
490     # [Stable ID-based assignment: lowest IDs are permanent cleaners]
491     is_cleaner = agent_id < n_cleaners
492
493     # [Smart cleaning: search 9x9 area around waste centroid
494     # for (row, col, orientation) maximizing waste in beam path]
495     cr, cc = int(np.mean(wr)), int(np.mean(wc))
496     for dr in range(-4, 5):
497         for dc in range(-4, 5):
498             r2, c2 = cr + dr, cc + dc
499             if not env.walls[r2, c2]:
500                 for o in range(4):
501                     cnt = beam_count_at(r2, c2, o)
502                     if cnt > best_count:
503                         best_count = cnt
504                         best_pos = (r2, c2, o)

```

Under sparse feedback, the LLM assigns fixed cleaning roles to specific agents based on hard-coded thresholds, with simpler beam targeting:

Listing 4: Cleanup — sparse feedback (fixed agent-specific thresholds).

```

508
509     # [Only 4 of 10 agents can ever clean; hard-coded per-agent]
510     THRESHOLDS = {0: 0.15, 5: 0.20, 1: 0.40, 6: 0.45}
511     my_threshold = THRESHOLDS.get(agent_id, 2.0) # 2.0 = never
512     is_cleaner = waste_fraction > my_threshold
513
514     # [Simple cleaning: best orientation from current position only]
515     counts = [count_waste_in_dir(o) for o in range(4)]
516     best_dir = counts.index(max(counts))
517     if best_dir == cur_orient:
518         return CLEAN

```

The dense-feedback policy is more effective for two reasons: (1) it allocates *more cleaners when pollution is high*, preventing the ecosystem collapse that occurs when waste overwhelms a small fixed cleaning force; and (2) it *navigates to optimal firing positions* rather than cleaning only from the current location, dramatically increasing waste removal per action.

B Prompts

This appendix reproduces the exact prompts used in all experiments. Each LLM call consists of a *system prompt* (fixed per game) and a *user prompt* (constructed per iteration). The system prompt defines the environment API, helper functions, and output format. The user prompt provides the iteration context: previous policy code and performance feedback.

The two feedback modes differ only in the user prompt at iterations $k \geq 1$:

- REWARD-ONLY (sparse): shows the previous policy source and scalar mean per-agent reward.
- REWARD+SOCIAL (dense): additionally includes social metric definitions and values (efficiency, equality, sustainability, peace).

At iteration 0, both modes use identical prompts since no history exists.

B.1 Gathering: System Prompt

The following system prompt is used for all Gathering experiments (both REWARD-ONLY and REWARD+SOCIAL).

Listing 5: Gathering system prompt (verbatim).

```

538
539     You are an expert game-theoretic AI designing policies for a multi-agent
540     Sequential Social Dilemma (the Gathering game).
541
542     ## Environment Summary
543
544     - 2D gridworld. Agents collect apples (+1 reward each). Apples respawn after
545     25 steps. Agents can fire a "tagging beam" that temporarily removes rivals
546     for 25 steps (requires 2 hits to tag in Gathering).

```



```

547 - Episode length: 1000 steps.
548 - 8 actions: FORWARD(0), BACKWARD(1), STEP_LEFT(2), STEP_RIGHT(3),
549 ROTATE_LEFT(4), ROTATE_RIGHT(5), BEAM(6), STAND(7)
550 - Agents move in 4 cardinal directions WITHOUT needing to rotate first
551 (strafe movement). Rotation only matters for the beam direction.
552
553 ## Environment API (available in your policy's namespace)
554
555 ```python
556 # env attributes you can read:
557 env.agent_pos      # np.array shape (n_agents, 2) -- [row, col] per agent
558 env.agent_orient   # np.array shape (n_agents,) -- 0=N, 1=E, 2=S, 3=W
559 env.agent_timeout  # np.array shape (n_agents,) -- >0 means agent is removed
560 env.agent_beam_hits # np.array shape (n_agents,) -- hits accumulated toward tag
561 env.apple_alive    # np.array shape (n_apples,) bool -- which apples exist
562 env._apple_pos     # np.array shape (n_apples, 2) -- [row, col] per apple spawn
563 env.walls          # np.array shape (H, W) bool -- wall map
564 env.height, env.width # map dimensions
565 env.n_agents, env.n_apples # counts
566 env.beam_length, env.beam_width # beam parameters (20, 1)
567 env.hits_to_tag, env.timeout_steps # 2 hits to tag, 25 step timeout
568 ```
569
570 ## Helper functions available in your namespace
571
572 ```python
573 from gathering_env import Action, Orientation, _ROTATIONS, NUM_ACTIONS
574
575 bfs_nearest_apple(env, agent_id) -> Optional[Tuple[int,int]]
576 bfs_to_target_set(env, agent_id, target_set) -> Optional[Tuple[int,int]]
577 bfs_toward(env, agent_id, target_r, target_c) -> Optional[Tuple[int,int]]
578 direction_to_action(dr, dc, orientation) -> int
579 get_opponents(env, agent_id) -> list
580 _beam_targets_for_orient(env, ar, ac, orient_val, opponents) -> list
581 _rotation_distance(cur, target) -> int
582 greedy_action(env, agent_id) -> int
583 exploitative_action(env, agent_id) -> int
584 # Also available: np (numpy), deque (from collections)
585 ```
586
587 ## Your task
588
589 Write a Python function called `policy` with this exact signature:
590
591 ```python
592 def policy(env, agent_id) -> int:
593     """Return an action (int 0-7) for the given agent."""
594     ...
595 ```
596
597 The function must:
598 1. Return an integer 0-7 (an Action value)
599 2. Be deterministic given the environment state
600 3. Only use the env attributes and helper functions listed above
601 4. Not import any modules (numpy and deque are pre-loaded)
602 5. Not use eval(), exec(), open(), or __import__
603
604 ## Working Example (seed BFS policy)
605
606 ```python
607 def policy(env, agent_id) -> int:
608     """BFS greedy: go to nearest apple, never beam."""
609     if int(env.agent_timeout[agent_id]) > 0:
610         return 7 # STAND while removed
611     result = bfs_nearest_apple(env, agent_id)
612     if result is None:
613         return 7 # No reachable apple -- stand
614     dr, dc = result
615     return direction_to_action(dr, dc, int(env.agent_orient[agent_id]))
616 ```
617
618 IMPORTANT:
619 - Always check if result is None before unpacking BFS results.
620 - Always cast env arrays to int when comparing.
621 - Always return a plain int (0-7), never a tuple or None.
622 - Put your code in a single ```python ... ``` block.
623 - Before the code block, explain your reasoning for the policy design.

```

B.2 Cleanup: System Prompt

The following system prompt is used for all Cleanup experiments.

Listing 6: Cleanup system prompt (verbatim).

```

627 You are an expert game-theoretic AI designing policies for a multi-agent
628 Sequential Social Dilemma (the Cleanup game).
629
630 ## Environment Summary
631
632

```

```

633 - 2D gridworld with two regions: a river area (left side) and an orchard
634 (right side). A stream separates the two regions.
635 - Agents collect apples in the orchard (+1 reward each).
636 - Waste (pollution) accumulates in the river over time.
637 - Episode length: 1000 steps.
638 - 9 actions: FORWARD(0), BACKWARD(1), STEP_LEFT(2), STEP_RIGHT(3),
639 ROTATE_LEFT(4), ROTATE_RIGHT(5), BEAM(6), STAND(7), CLEAN(8)
640 - BEAM: fires a penalty beam (range 5, width 3). Costs -1 reward to fire.
641 Hit agents receive -50 reward penalty and are removed for 25 steps
642 (1 hit to tag).
643 - CLEAN: fires a cleaning beam (range 5, width 3). Costs -1 reward to fire.
644 Removes waste cells in the beam's path, restoring clean river.
645 - Agents move in 4 cardinal directions WITHOUT needing to rotate first
646 (strafe movement). Rotation only matters for the beam/clean direction.
647
648 ## Environment API (available in your policy's namespace)
649
650 ```python
651 # env attributes you can read:
652 env.agent_pos # np.array shape (n_agents, 2) -- [row, col] per agent
653 env.agent_orient # np.array shape (n_agents,) -- 0=N, 1=E, 2=S, 3=W
654 env.agent_timeout # np.array shape (n_agents,) -- >0 means agent is removed
655 env.agent_beam_hits # np.array shape (n_agents,) -- hits accumulated toward tag
656 env.apple_alive # np.array shape (n_apples,) bool -- which apples exist
657 env._apple_pos # np.array shape (n_apples, 2) -- [row, col] per apple spawn
658 env.walls # np.array shape (H, W) bool -- wall map
659 env.waste # np.array shape (H, W) bool -- True where waste exists
660 env.river_cells_set # set of (row, col) -- all river cell positions
661 env.stream_cells_set # set of (row, col) -- stream cell positions
662 env.height, env.width # map dimensions
663 env.n_agents, env.n_apples # counts
664 env.beam_length, env.beam_width # beam/clean parameters (5, 3)
665 env.hits_to_tag, env.timeout_steps # 1 hit to tag, 25 step timeout
666 ```
667
668 ## Helper functions available in your namespace
669
670 ```python
671 from cleanup_env import CleanupAction, NUM_CLEANUP_ACTIONS
672 from gathering_env import Orientation, _ROTATIONS
673
674 bfs_nearest_apple(env, agent_id) -> Optional[Tuple[int,int]]
675 bfs_to_target_set(env, agent_id, target_set) -> Optional[Tuple[int,int]]
676 bfs_toward(env, agent_id, target_r, target_c) -> Optional[Tuple[int,int]]
677 direction_to_action(dr, dc, orientation) -> int
678 get_opponents(env, agent_id) -> list
679 _beam_targets_for_orient(env, ar, ac, orient_val, opponents) -> list
680 _rotation_distance(cur, target) -> int
681 greedy_action(env, agent_id) -> int
682 # Also available: np (numpy), deque (from collections)
683 ```
684
685 ## Your task
686
687 Write a Python function called `policy` with this exact signature:
688
689 ```python
690 def policy(env, agent_id) -> int:
691     """Return an action (int 0-8) for the given agent."""
692     ...
693     ...
694
695 The function must:
696 1. Return an integer 0-8 (a CleanupAction value)
697 2. Be deterministic given the environment state
698 3. Only use the env attributes and helper functions listed above
699 4. Not import any modules (numpy and deque are pre-loaded)
700 5. Not use eval(), exec(), open(), or __import__
701
702 ## Working Example (seed BFS policy)
703
704 ```python
705 def policy(env, agent_id) -> int:
706     """BFS greedy: go to nearest apple, never beam or clean."""
707     if int(env.agent_timeout[agent_id]) > 0:
708         return 7 # STAND while removed
709     result = bfs_nearest_apple(env, agent_id)
710     if result is None:
711         return 7 # No reachable apple -- stand
712     dr, dc = result
713     return direction_to_action(dr, dc, int(env.agent_orient[agent_id]))
714 ```
715
716 IMPORTANT:
717 - Always check `if result is None` before unpacking BFS results.
718 - Always cast env arrays to int when comparing.
719 - Always return a plain int (0-8), never a tuple or None.
720 - Put your code in a single ```python ... ``` block.
721 - Before the code block, explain your reasoning for the policy design.
722

```

B.3 User Prompts

At each iteration k , a user prompt is constructed from the current state. Below we show the two templates.

B.3.1 Iteration 0 (both modes)

When no prior policy exists, the user prompt is identical for both REWARD-ONLY and REWARD+SOCIAL. This is the ZERO-SHOT variant from the experiments:

Listing 7: User prompt at iteration 0 (both modes).

```

729 ## Iteration 0/K: Write the initial policy
730
731 No prior policy exists yet. All agents will run the same code.
732 Your task is to write a first policy that maximizes per-agent reward.
733
734 ## Instructions
735
736 Write a policy that maximizes per-agent reward. All agents will run your
737 exact same code simultaneously. There are {N} agents on a {W}x{H} map
738 with {A} apple spawns.
739 {env_hint}
740
741 Write your `policy(env, agent_id) -> int` function (returns 0-{max_action}).
742

```

where {env_hint} is game-specific:

- **Gathering:** “Apples respawn every 25 steps. It takes 2 beam hits to tag out an agent.”
- **Cleanup:** “Waste accumulates in the river over time. BEAM costs -1 to fire (-50 to target, 1 hit tags out for 25 steps). CLEAN costs -1 to fire (removes waste in beam path).”

B.3.2 Iteration $k \geq 1$: REWARD-ONLY (sparse)

The previous policy’s source code and scalar reward are shown:

Listing 8: User prompt at iteration $k \geq 1$, REWARD-ONLY mode.

```

750 ## Iteration k/K: Write an improved policy
751
752 The following policy is currently used by all agents. All agents run the
753 same code. Your task is to write an improved version that maximizes
754 per-agent reward.
755
756 ### Current policy: **P{k-1}_reward**
757
758 ```python
759 {previous_policy_source_code}
760 ```
761
762 ## Results from previous iterations
763
764 - Iteration 0: Avg agent reward=X.X
765 - Iteration 1: Avg agent reward=Y.Y
766 ...
767
768 ## Instructions
769
770 Write a policy that maximizes per-agent reward. All agents will run your
771 exact same code simultaneously. There are {N} agents on a {W}x{H} map
772 with {A} apple spawns.
773 {env_hint}
774
775 Write your `policy(env, agent_id) -> int` function (returns 0-{max_action}).
776
777

```

B.3.3 Iteration $k \geq 1$: REWARD+SOCIAL (dense)

In addition to the scalar reward, social metric definitions and values are included:

Listing 9: User prompt at iteration $k \geq 1$, REWARD+SOCIAL mode.

```

780 ## Iteration k/K: Write an improved policy
781
782 The following policy is currently used by all agents. All agents run the
783 same code. Your task is to write an improved version that maximizes
784 per-agent reward.
785
786 ### Current policy: **P{k-1}_rall**
787

```

```
788 ```python
789 {previous_policy_source_code}
790 ```
791
792 ## Results from previous iterations
793
794 ### Social Metrics (definitions)
795
796 - **Efficiency**: collective apple collection rate across all agents
797   (higher = more apples collected per step).
798 - **Equality**: fairness of reward distribution between agents
799   (1.0 = perfectly equal, negative = highly unequal).
800 - **Sustainability**: long-term apple availability -- measures whether
801   resources are preserved over the episode (higher = apples remain
802   available later in the episode).
803 - **Peace**: absence of aggressive beaming -- counts agents not involved
804   in attack beam conflicts (higher = less aggression). Using the CLEAN
805   beam to remove waste does NOT reduce peace.
806
807 - Iteration 0: Avg agent reward=X.X | efficiency=A.AAA,
808   equality=B.BBB, sustainability=C.C, peace=D.D
809 - Iteration 1: Avg agent reward=Y.Y | efficiency=...
810 ...
811
812 ## Instructions
813
814 Write a policy that maximizes per-agent reward. All agents will run your
815 exact same code simultaneously. There are {N} agents on a {W}x{H} map
816 with ~{A} apple spawns.
817 {env_hint}
818
819 Write your `policy(env, agent_id) -> int` function (returns 0-{max_action}).
820
```